QUY TRÌNH MÃ HÓA ĐẦU CUỐI E2EE TRONG VINASOCIAL

Mô tả theo mã nguồn và tài liệu hiện có trong repository VinaSocial • 06/11/2026

Phạm vi. Tài liệu giải thích E2EE của tin nhắn website: tạo và bảo vệ khóa thiết bị, gửi văn bản/sticker, gửi ảnh/video/âm thanh/tệp, đọc tin, ghép nối thiết bị và chuyển lịch sử. Đây là mô tả của bản triển khai prototype; không phải chứng nhận bảo mật hoặc kết quả kiểm toán độc lập.

1. Ba loại khóa và vai trò

Loại khóa	Vai trò	Nơi lưu / tồn tại
Cặp RSA-OAEP của thiết bị	Public key bọc khóa AES cho thiết bị; private key mở wrapper.	Public key ở server. Private key được mã hóa trong IndexedDB và mở trong bộ nhớ tab.
Khóa AES-256 của tin	Mã hóa payload JSON của một lần gửi hoặc sửa tin.	Tạo mới trong client. Server chỉ lưu bản được bọc bằng RSA cho từng thiết bị.
Khóa AES-256 của tệp	Mã hóa bytes của một tệp đính kèm.	Tạo mới trong client. Khóa và IV được đặt trong payload tin đã mã hóa.

Điểm dễ nhầm: Argon2id là hàm dẫn xuất khóa từ mật khẩu mở kho. Nó không trực tiếp mã hóa private key hay nội dung tin nhắn; AES-GCM thực hiện hai việc đó ở các bước khác nhau.

2. Tạo thiết bị và bảo vệ private key

- **Khởi tạo.** Một tài khoản trên một browser profile tạo một cặp RSA-OAEP 3072 bit, public exponent 65537, SHA-256 và một deviceId UUID. Public key xuất dạng JWK để đăng ký với RelationshipService.
- **Bọc private key để lưu.** Trình duyệt tạm export private key PKCS#8 trong bộ nhớ, sinh salt 16 byte và IV 12 byte. Argon2id dẫn xuất 32 byte từ mật khẩu mở kho (hiện dùng 19 MiB, 2 vòng, parallelism 1). Khóa dẫn xuất được import làm khóa AES-256-GCM để mã hóa PKCS#8.
- **Gắn ngữ cảnh.** AAD của lớp bảo vệ private key gồm userId, deviceId và public JWK. Thay đổi các thông tin này hoặc ciphertext làm kiểm tra AES-GCM thất bại.
- **Lưu cục bộ.** IndexedDB vinasocial-e2ee, store device-keys, lưu private key đã mã hóa, salt, IV, tham số KDF, public JWK và metadata thiết bị. Mật khẩu và đầu ra Argon2id dạng rõ không được lưu hoặc gửi server.
- **Mở kho.** Sau khi tải lại trang hoặc khôi phục phiên, người dùng nhập mật khẩu. Client chạy Argon2id với salt đã lưu, giải mã PKCS#8 và import private key với extractable=false để dùng trong tab. Sai mật khẩu hoặc dữ liệu hỏng sẽ không mở được kho.

Khi tạo mới, private key có thể export tạm để bọc tại chỗ; sau đó bản dùng lúc chạy được import lại không cho export. Đăng xuất/đổi tài khoản hoặc đóng/tải lại tab khóa bản trong bộ nhớ. Bản mã IndexedDB vẫn còn. Các bản ghi CryptoKey legacy được đọc để giữ khả năng giải mã nhưng không tự chuyển thành dạng có mật khẩu.

3. Gửi nội dung: văn bản, emoji và sticker

- **Kiểm tra chế độ.** Website đọc thiết lập E2EE theo tài khoản (mặc định bật). Nếu tắt, xem mục 7.

- **Xác định thiết bị nhận.** Với chat nhóm, website lấy danh sách thành viên; với chat trực tiếp dùng tài khoản người nhận. Nó thêm cả người gửi để các thiết bị của người gửi đọc lại tin. API chỉ cung cấp public key của thiết bị ACTIVE. Nếu một tài khoản chưa có thiết bị ACTIVE, gửi dừng.
- **Kiểm tra khóa.** Client ghi nhớ public key đã thấy theo deviceId (TOFU). Nếu cùng deviceId nay có userId hoặc public key khác, client báo lỗi. Kiểm tra này không phải xác minh fingerprint ngoài ứng dụng.
- **Chuẩn bị nội dung.** Client tạo clientMessageId UUID và JSON như {"type":"TEXT","content":"Xin chào"} hoặc {"type":"STICKER","content":"..."}. Emoji trong văn bản là một phần của content.
- **Mã hóa.** Mỗi lần tạo tin, client sinh một khóa AES-256 mới và IV ngẫu nhiên 12 byte; AES-GCM mã hóa JSON, tag 128 bit, AAD là chuỗi dưới đây. IV và AAD không cần bí mật nhưng phải đúng khi giải mã.

```
vinasocial:e2ee:v1:<clientMessageId>
```

- **Bọc khóa.** Client tạm export 32 byte khóa AES, rồi dùng RSA-OAEP/SHA-256 mã hóa bản khóa ấy riêng cho public key của từng thiết bị người nhận và người gửi.
- **Gửi.** Client gửi clientMessageId và envelope JSON. Trường type ở lớp API/lưu trữ là TEXT cho tin E2EE; type nội dung thực tế nằm trong payload đã mã hóa.

4. Envelope và điều server thực sự kiểm tra

```
{
  "version": 1,
  "algorithm": "AES-256-GCM+RSA-OAEP-256",
  "iv": "<Base64 IV>",
  "ciphertext": "<Base64 AES-GCM ciphertext + tag>",
  "keys": {
    "<device UUID>": "<Base64 AES key boc bang RSA-OAEP>"
  }
}
```

RelationshipService lưu envelope trong messages.ciphertext cùng clientMessageId, sender, conversation, thời gian, trạng thái và các metadata cần vận hành. Backend kiểm tra version, tên thuật toán, Base64, IV dài 12 byte, ciphertext có ít nhất 16 byte tag, deviceId UUID, RSA ciphertext dài 384 byte và tối đa 256 wrapper. Đây là kiểm tra **định dạng**; backend không giải mã plaintext và không chứng minh đã có đủ wrapper cho mọi thiết bị của mọi thành viên.

AES-GCM dùng AES theo chế độ bộ đếm để che nội dung, đồng thời dùng GHASH để xác thực ciphertext và AAD. Thay đổi ciphertext, IV hoặc clientMessageId khiến bước xác thực thất bại. RSA-OAEP chỉ mã hóa khóa AES nhỏ, không mã hóa trực tiếp cả tin nhắn hay bytes tệp.

5. Gửi ảnh, video, âm thanh và tệp

Tệp dùng **hai khóa AES khác nhau**: một khóa mã hóa bytes tệp; khóa còn lại mã hóa payload tin nhắn chứa thông tin cần để mở tệp.

- Client tạo clientMessageId, phân loại IMAGE / VIDEO / AUDIO / FILE theo MIME type, rồi sinh khóa AES-256 và IV 12 byte riêng cho tệp.
- Client đọc bytes tệp, dùng AES-GCM mã hóa với AAD:

```
vinasocial:e2ee:v1:<clientMessageId>:media
```

- Client tạo tệp ciphertext tên UUID.enc, MIME application/octet-stream, rồi upload với isEncrypt=true, isE2EE=true và purpose=MESSAGE. MediaService/Cloudinary lưu bytes ciphertext; MediaService không lưu AES key hoặc IV của tệp.

- Sau khi API upload trả objectKey, client tạo payload tin có type thực tế và media gồm objectKey, key (Base64 khóa AES tệp), iv, contentType gốc và fileName gốc. Toàn bộ payload này được mã hóa như tin nhắn ở mục 3 bằng **khóa AES của tin**.

```
{
  "type": "IMAGE | VIDEO | AUDIO | FILE",
  "media": {
    "objectKey": "<dia chi ciphertext>",
    "key": "<Base64 khoa AES cua tep>",
    "iv": "<Base64 IV cua tep>",
    "contentType": "<MIME goc>",
    "fileName": "<ten goc>"
  }
}
```

Upload tệp và gửi tin là hai thao tác riêng. Nếu upload thành công nhưng gửi tin thất bại, object ciphertext có thể đã tồn tại mà chưa có message tham chiếu. MediaService vẫn thấy người tải lên, thời điểm, kích thước gần đúng, purpose MESSAGE và trạng thái E2EE.

6. Người nhận đọc tin và mở tệp

- Website tải lịch sử gồm clientMessageId, ciphertext envelope và wrapper lịch sử bổ sung (nếu có). Tin DELETED được hiển thị là đã xóa, không giải mã.
- Client dùng private key RSA của thiết bị hiện tại. Nếu kho protected đang khóa, người dùng phải nhập mật khẩu mở kho để client phục hồi private key trong bộ nhớ.
- Client chọn keys[deviceId] trong envelope; nếu không có thì thử wrapper lịch sử của cùng deviceId. Không có wrapper phù hợp thì thiết bị không đọc được tin đó.
- RSA-OAEP dùng private key mở wrapper để thu lại khóa AES của tin. AES-GCM dùng khóa này, IV, ciphertext và AAD gắn clientMessageId để xác thực tag và giải mã JSON. Client kiểm tra cấu trúc payload rồi mới hiển thị. Lỗi ở bất kỳ bước nào được giao diện báo là không thể giải mã trên thiết bị này.
- Nếu payload chứa media, client lấy objectKey, khóa và IV tệp từ JSON đã giải mã, tải bytes ciphertext, dùng AES-GCM với AAD có hậu tố :media để xác thực và giải mã bytes, rồi tạo Blob URL trong trình duyệt để hiển thị hoặc tải tệp.

Tin rõ cũ có nhánh đọc tương thích. Việc ứng dụng hiển thị được một tin không tự chứng minh rằng tin ấy từng được gửi bằng E2EE.

7. Khi tắt E2EE và khi sửa tin

Thiết lập E2EE theo tài khoản mặc định bật nhưng có thể tắt. Khi tắt, văn bản/sticker và objectKey của tệp gửi dạng rõ; file gốc được upload mà không chạy encryptAttachment. Backend kiểm tra nội dung gửi có khớp chế độ hiện tại. Tin cũ giữ nguyên trạng thái lúc gửi. Khi bật lại, người dùng phải mở/tạo kho khóa để gửi tin mã hóa.

Sửa tin E2EE tạo ciphertext và khóa AES mới nhưng vẫn dùng clientMessageId ban đầu làm AAD. Vì ciphertext đổi, wrapper chuyển lịch sử gắn với fingerprint ciphertext cũ không còn hiệu lực và có thể cần chuyển lại.

8. Thiết bị mới, chuyển lịch sử và thu hồi

Ghép nối. Thiết bị E2EE đầu tiên của tài khoản được ACTIVE. Thiết bị mới đăng ký sau đó ở PENDING_APPROVAL, không có trong danh sách public key dùng gửi tin. Một phiên đã liên kết với thiết bị ACTIVE duyệt yêu cầu; server cấp mã 8 chữ số, dùng một lần, hết hạn sau 3 phút và giới hạn số lần nhập sai. Thiết bị mới nhập mã để thành ACTIVE. Mã ghép nối không phải khóa mã hóa tin.

Lịch sử. Kích hoạt chỉ giúp thiết bị mới nhận tin gửi sau đó. Muốn đọc tin cũ, thiết bị nguồn đã mở kho tải từng tin, mở khóa AES từ wrapper của mình rồi bọc lại khóa AES ấy bằng public key thiết bị đích. Client gửi wrapper mới cùng SHA-256 fingerprint của ciphertext lên server. Quá trình này không gửi plaintext tin hay private key. Thiết bị đích chỉ mở được những tin mà thiết bị nguồn có khóa phù hợp và đã chuyển wrapper.

Thu hồi. DeviceId bị thu hồi không còn được chọn để bọc khóa tin mới. Thu hồi không xóa ciphertext cũ, không xóa private key đã có trên thiết bị và không ngăn thiết bị ấy đọc lại nội dung mà nó đã nhận được trước đó.

9. Mức bảo vệ và các giới hạn cần nói rõ

- Với website chạy đúng luồng và E2EE bật, server nhận ciphertext của tin và tệp, cùng các khóa AES đã bọc; không nhận private key hoặc khóa AES tin dạng rõ.
- Server vẫn thấy metadata như người gửi, người nhận, thời điểm, trạng thái và kích thước gần đúng. E2EE không che nội dung trên thiết bị đã mở khóa, khỏi XSS/mã độc/extension hoặc khỏi người nhận sao chép.
- Thiết kế hiện tại không có forward secrecy, Double Ratchet hay phục hồi sau khi khóa thiết bị bị lộ. Một private key bị lộ có thể mở những tin mà thiết bị đó có wrapper và kẻ tấn công có ciphertext tương ứng.
- Chưa có fingerprint/safety number để người dùng tự xác minh public key qua kênh độc lập. TOFU chỉ phát hiện thay đổi khóa dưới cùng deviceId đã thấy; server bị chiếm quyền hoặc client độc hại nằm ngoài bảo đảm của mô tả này.
- Nếu mất browser storage hoặc quên mật khẩu mở kho và không còn thiết bị khác có thể chuyển lịch sử, dự án chưa có recovery key độc lập. Backend chỉ kiểm tra envelope có cấu trúc hợp lệ, không thể xác minh payload đã thực sự được mã hóa đúng bởi client tùy ý.

10. Đường dẫn mã nguồn đối chiếu

docs/E2EE-IMPLEMENTATION.md - tài liệu thiết kế, luồng và giới hạn

client/website/src/utils/e2eeMessageUtil.ts - vault, envelope, mã hóa/giải mã tin và tệp

client/website/src/utils/outgoingMessageUtil.ts - áp dụng thiết lập E2EE khi gửi

client/website/src/hooks/useDecryptedMessages.ts - giải mã lịch sử để hiển thị

client/website/src/components/social/MessagesComponent.tsx - luồng upload trong chat

server/RelationshipService/src/main/java/vn/tuhoc/RelationshipService/utils/E2eeEnvelopeValidationUtil.java - kiểm tra envelope

server/RelationshipService/src/main/java/vn/tuhoc/RelationshipService/services/MessageEncryptionSettingService.java - kiểm tra chế độ

Tham khảo thuật toán: NIST FIPS 197 (AES), NIST SP 800-38D (GCM), RFC 8017 (RSA-OAEP), RFC 9106 (Argon2id).